

AI Research Assistant

Guia gratuita paso a paso para construir un proyecto completo de AI Engineer

Proyecto: RAG + PDFs + Vector Database + LLM gratis + React + FastAPI

Objetivo del documento

Construir una aplicacion capaz de subir PDFs, extraer texto, crear embeddings, guardar informacion en una base vectorial local y responder preguntas citando las fuentes encontradas. La version inicial se ejecuta localmente y usa herramientas gratuitas o open-source.

Resultado final esperado

- Frontend en React para subir PDFs y hacer preguntas.
- Backend en FastAPI con endpoints /upload y /chat.
- ChromaDB local como vector database.
- Embeddings locales con sentence-transformers.
- LLM gratuito mediante Groq.
- Respuestas con fuentes: nombre del archivo y pagina.

1. Stack tecnologico gratuito

Area	Herramienta	Motivo
Frontend	React + Vite	Rapido, simple y facil de mostrar en portfolio.
Backend	Python + FastAPI	Muy usado en AI Engineering y facil para APIs.
LLM	Groq	Tiene acceso gratuito a modelos open-source con limites.
Vector DB	ChromaDB local	Open-source y simple para RAG.
Embeddings	sentence-transformers	Corre localmente sin pagar API.
PDF	PyMuPDF	Extrae texto de PDFs de forma simple.
Agentes	LangGraph	Opcional para version avanzada.

2. Arquitectura general

```
Frontend React
-> FastAPI Backend
-> PDF Loader
-> Embeddings locales
-> ChromaDB
-> Retrieval
-> Prompt con contexto
-> LLM en Groq
-> Respuesta con fuentes
```

La idea central es RAG: el modelo no responde solamente con conocimiento general. Primero recuperamos fragmentos relevantes de documentos y luego se los pasamos al LLM como contexto.

3. Estructura del proyecto

```
ai-research-assistant/
|-- backend/
| |-- app/
| | |-- main.py
| | |-- rag.py
| | |-- pdf_loader.py
| | |-- llm.py
| | `-- config.py
|-- uploads/
|-- chroma_db/
|-- requirements.txt
|-- .env
`-- frontend/
```

4. Crear el backend

Desde una terminal:

```
mkdir ai-research-assistant
cd ai-research-assistant
mkdir backend
cd backend
python -m venv venv
```

Activar entorno virtual en Windows:

```
venv\Scripts\activate
```

Activar entorno virtual en Linux/Mac:

```
source venv/bin/activate
```

Instalar dependencias:

```
pip install fastapi uvicorn python-dotenv pymupdf chromadb sentence-transformers groq
python-multipart
pip freeze > requirements.txt
```

5. Variables de entorno

Crear el archivo backend/.env:

```
GROQ_API_KEY=tu_api_key_de_groq
GROQ_MODEL=llama-3.1-8b-instant
```

Nota: no subas el archivo .env a GitHub. Agrega .env al .gitignore.

6. Código del backend

Crear carpeta app dentro de backend y agregar los siguientes archivos.

app/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

GROQ_API_KEY = os.getenv("GROQ_API_KEY")
GROQ_MODEL = os.getenv("GROQ_MODEL", "llama-3.1-8b-instant")
```

app/llm.py

```
from groq import Groq
from app.config import GROQ_API_KEY, GROQ_MODEL

client = Groq(api_key=GROQ_API_KEY)

def ask_llm(prompt: str) -> str:
    response = client.chat.completions.create(
        model=GROQ_MODEL,
        messages=[
            {"role": "system", "content": "Eres un asistente de investigacion. Responde con precision y cita fuentes."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.2
    )
    return response.choices[0].message.content
```

app/pdf_loader.py

```
import fitz

def extract_text_from_pdf(file_path: str) -> list[dict]:
    doc = fitz.open(file_path)
    pages = []

    for page_number, page in enumerate(doc, start=1):
        text = page.get_text()
        if text.strip():
            pages.append({"page": page_number, "text": text})

    return pages
```

app/rag.py

```
import chromadb
from sentence_transformers import SentenceTransformer

client = chromadb.PersistentClient(path="chroma_db")
collection = client.get_or_create_collection(name="documents")
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

def chunk_text(text: str, chunk_size: int = 800, overlap: int = 150):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks

def add_document(filename: str, pages: list[dict]):
    ids, documents, metadatas, embeddings = [], [], [], []
    counter = 0
    for page in pages:
        for chunk in chunk_text(page["text"]):
            doc_id = f"{filename}-page-{page['page']}-chunk-{counter}"
            ids.append(doc_id)
            documents.append(chunk)
            metadatas.append({"filename": filename, "page": page["page"]})
            embeddings.append(embedding_model.encode(chunk).tolist())
            counter += 1
    collection.add(ids=ids, documents=documents, metadatas=metadatas, embeddings=embeddings)

def search_documents(query: str, n_results: int = 5):
    query_embedding = embedding_model.encode(query).tolist()
    results = collection.query(query_embeddings=[query_embedding], n_results=n_results)
    docs = []
    for i in range(len(results["documents"][0])):
        docs.append({"text": results["documents"][0][i], "metadata": results["metadatas"][0][i]})
    return docs
```

app/main.py

```
import os
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from app.pdf_loader import extract_text_from_pdf
from app.rag import add_document, search_documents
from app.llm import ask_llm

app = FastAPI(title="AI Research Assistant")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.get("/")
def root():
    return {"message": "AI Research Assistant API"}

@app.post("/upload")
async def upload_pdf(file: UploadFile = File(...)):
    file_path = os.path.join(UPLOAD_DIR, file.filename)
    with open(file_path, "wb") as f:
        f.write(await file.read())

    pages = extract_text_from_pdf(file_path)
    add_document(file.filename, pages)

    return {"message": "PDF procesado correctamente", "filename": file.filename, "pages":
        len(pages)}

@app.post("/chat")
async def chat(payload: dict):
    question = payload.get("question", "")
    docs = search_documents(question)
    context = "\n\n".join([
        f"Fuente: {doc['metadata']['filename']} - Pagina {doc['metadata']['page']}\n{doc['text']}"
        for doc in docs
    ])

    prompt = f"""
    Responde usando el siguiente contexto.
    Si no encuentras la respuesta en el contexto, dilo claramente.

    Contexto:
    {context}

    Pregunta:
    {question}

    Incluye fuentes al final.
    """
    answer = ask_llm(prompt)
    return {"answer": answer, "sources": [doc["metadata"] for doc in docs]}
```

Ejecutar backend

```
uvicorn app.main:app --reload
```

Luego abrir <http://localhost:8000/docs> para probar los endpoints desde Swagger.

7. Crear el frontend

```
cd ..
npm create vite@latest frontend
cd frontend
npm install
npm install axios
npm run dev
```

Reemplazar src/App.jsx por:

```
import { useState } from "react";
import axios from "axios";
import "../App.css";

const API_URL = "http://localhost:8000";

function App() {
  const [file, setFile] = useState(null);
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");
  const [sources, setSources] = useState([]);
  const [loading, setLoading] = useState(false);

  const uploadPdf = async () => {
    if (!file) return alert("Selecciona un PDF");
    const formData = new FormData();
    formData.append("file", file);
    setLoading(true);
    try {
      await axios.post(`${API_URL}/upload`, formData);
      alert("PDF cargado correctamente");
    } catch {
      alert("Error al subir PDF");
    } finally {
      setLoading(false);
    }
  };

  const askQuestion = async () => {
    if (!question.trim()) return;
    setLoading(true);
    setAnswer("");
    setSources([]);
    try {
      const response = await axios.post(`${API_URL}/chat`, { question });
      setAnswer(response.data.answer);
      setSources(response.data.sources || []);
    } catch {
      setAnswer("Error al consultar la IA");
    } finally {
      setLoading(false);
    }
  };

  return (
    <main style={{ maxWidth: "900px", margin: "40px auto", fontFamily: "Arial" }}>
      <h1>AI Research Assistant</h1>
      <h2>Subir PDF</h2>
      <input type="file" accept="application/pdf" onChange={(e) => setFile(e.target.files[0])} />
      <button onClick={uploadPdf} disabled={loading}>Subir PDF</button>

      <h2>Chat</h2>
      <textarea rows="4" style={{ width: "100%" }} value={question}
        onChange={(e) => setQuestion(e.target.value)}
        placeholder="Pregunta algo sobre tus documentos..." />
      <button onClick={askQuestion} disabled={loading}>Preguntar</button>

      {loading && <p>Procesando...</p>}
      {answer && <div style={{ whiteSpace: "pre-wrap" }}><h3>Respuesta</h3><p>{answer}</p></div>}
      {sources.length > 0 && (
        <div><h3>Fuentes</h3><ul>{sources.map((s, i) => (
          <li key={i}>{s.filename} - Pagina {s.page}</li>

```

```
    )})</ul></div>  
  })  
</main>  
);  
}  
  
export default App;
```

8. Demo para entrevista

- Iniciar backend con uvicorn.
- Iniciar frontend con npm run dev.
- Subir un PDF real.
- Hacer una pregunta especifica sobre el PDF.
- Mostrar que la respuesta incluye fuentes.
- Explicar que el sistema recupera chunks relevantes antes de llamar al LLM.
- Explicar que esto reduce hallucinations porque el modelo responde con contexto.

9. Checklist de version portfolio

- README con descripcion, arquitectura, screenshots e instrucciones.
- Archivo .env.example sin claves reales.
- Dockerfile para backend.
- docker-compose con backend y frontend.
- Historial de chats con PostgreSQL.
- Streaming de respuesta.
- Comparacion entre dos PDFs.
- Tests basicos para endpoints.
- Deploy en un servicio gratuito o demo local grabada.

10. Como explicarlo en entrevista

Una forma simple de presentarlo:

Construí un AI Research Assistant con RAG. El usuario sube documentos, el backend extrae el texto, lo divide en chunks, genera embeddings locales, guarda esos vectores en ChromaDB y, cuando el usuario pregunta algo, recupera los fragmentos mas relevantes. Luego el LLM recibe solo ese contexto y responde citando las fuentes.

11. Orden recomendado de implementacion

1. Backend FastAPI basico.
2. Upload de PDF.
3. Extraccion de texto.
4. Chunking.
5. Embeddings.
6. ChromaDB.
7. Endpoint /chat.
8. Frontend React.
9. Fuentes en la respuesta.
10. README y demo.